A Box That Just Runs

Lecture 5 · Docker and containers

Images · containers · Dockerfiles · Compose · an AI that drafts, and Docker that judges

Slides, demos, exercise and labs: `lectures/05-docker/`

Today

Time	What	Where to find it
6 min	Tom's request, why Docker, images and containers	
22 min	**Steps 1 to 6**: run, build and ship a box	Demo guides 1 to 3
7 min	**Three Mentimeter questions** during the demos	menti.com
7 min	Habits, gotchas and the labs	
8 min	**Attendance form** and **anonymous feedback**	QR codes at the end

Everything I type is written up step by step in the demo guides. Follow along, or just watch.

Monday, 9:00: a message from Tom

> **Tom** (platform): "Your menu page works great on your laptop. Staging has **Python 3.9**, **no uv**, and I'm not allowed to change it. I'm not installing your laptop on my server."
>
> "Can you give me **a box that just runs**?"

Jordan: "Same lesson as the 3 AM Page: something was different over there. Pack the environment **with** the code."

By the end of today you can…

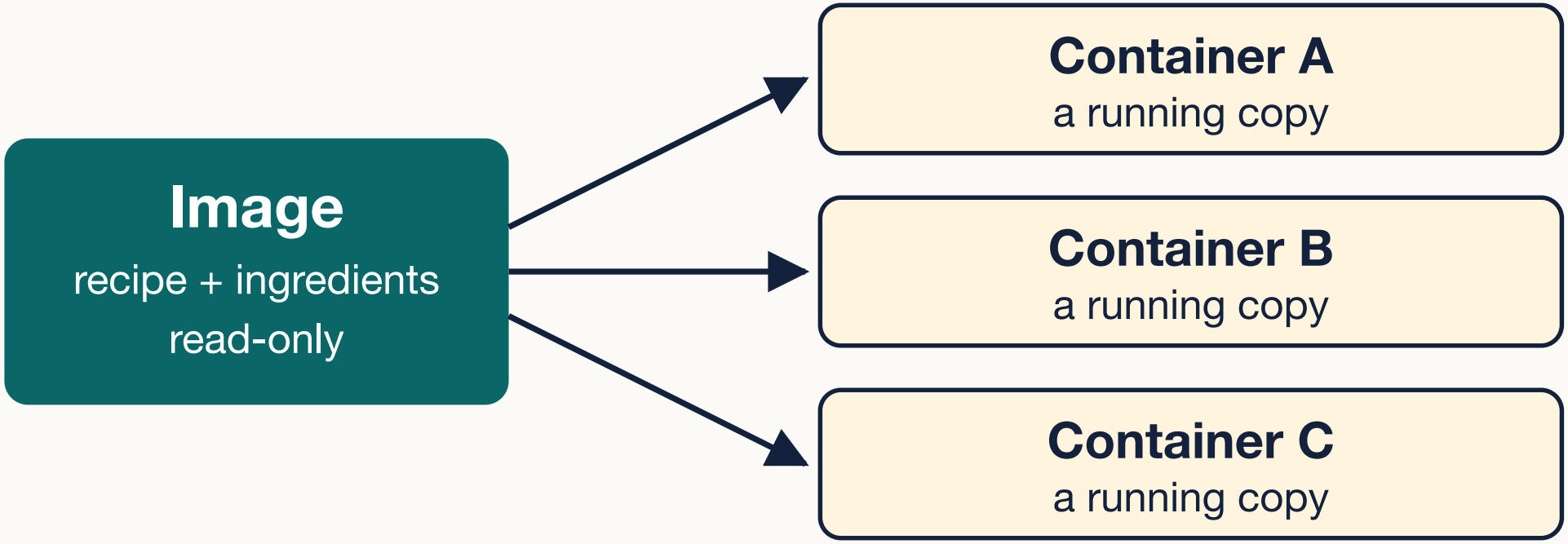
1. Explain **image**, **container** and how a container differs from a **virtual machine**
2. **Run, inspect, stop and clean up** containers (`run`, `ps`, `logs`, `exec`, `stop`, `rm`)
3. Read, and write, a **Dockerfile** for a Python service
4. Use a **log** to find out why a container won't start
5. Start an app **and its database** with one **Compose** command
6. Use an **AI** to draft, explain and review a Dockerfile, and **prove** each answer with Docker

Why bother? The evidence

Source	Finding
Stack Overflow Developer Survey, 2025	Docker used by **73.8%** of professional developers and **52.5%** of people learning to code
ITJobsWatch, UK permanent ads, 6 months to 3 Nov 2026	**2,046** ads mention Docker; median salary **£67,500**
SFIA Deployment, entry level (Level 2)	"Executes defined deployment processes and procedures using deployment tools and techniques."

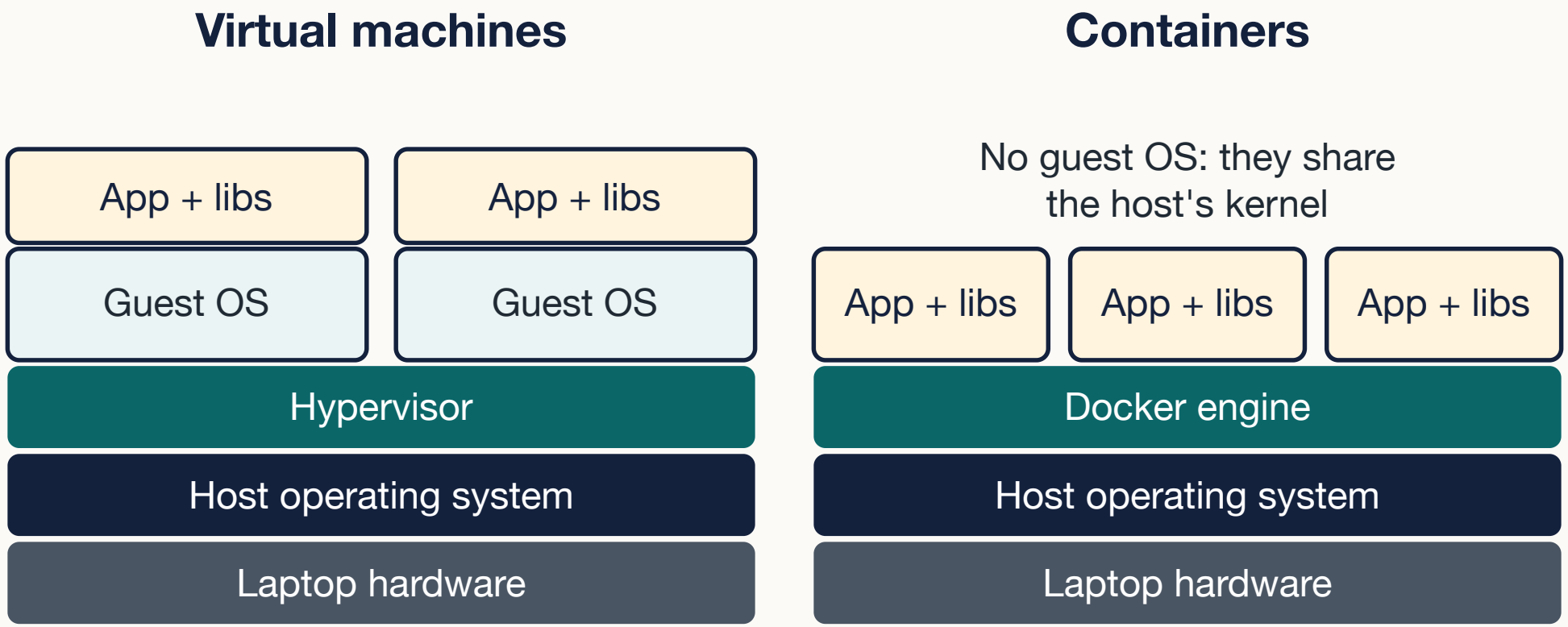
The gap between those two survey numbers is what today closes.

An image is the recipe, a container is the meal



An **image** is stored once and never changes. Each **container** is a running copy cooked from it.

Containers are not virtual machines



A VM brings **a whole kitchen**; a container shares yours. That's why it starts in a second.

Step 1: run someone else's software

```
docker version
docker run --rm hello-world
docker run --rm python:3.12-slim python -V
```

- `run` = create a container from an **image** and start it
- `--rm` = delete the container when it stops
- `python:3.12-slim` = the image **name:tag**; `python -V` = the command to run **inside**

Nothing was installed on this laptop for that.

Mentimeter question 1 of 3

Predict: you start **three containers from the same image**. One of them creates a new file. What do the **other two** containers see?

1. The new file, because they share one image
2. Nothing: each container has its own private changes
3. The new file, but only after a restart
4. It depends on how much memory they have

menti.com
code **2590 7185**

Question 1: the answer is 2, nothing

Option	Why
The new file, they share one image	The image is **read-only**: nobody can change it from a container
2. Nothing (correct)	**Each container writes to its own private layer**, which disappears with it
The new file, after a restart	Restarting doesn't merge containers
Depends on memory	Memory doesn't change what a container can see

> Anything that must survive a container (a database!) goes in a **volume**. We'll see one in Step 6.

Step 2: start a web server

```
docker run -d --name web -p 8080:80 nginx; echo http://localhost:8080
docker ps
docker logs web
docker exec -it web sh
```

Open **localhost:8080**, reload, run `docker logs web` again.

- `-d` background · `--name web` a name you choose
- `-p 8080:80` = **my door 8080 → the box's door 80** (always **host:container**)
- `exec -it … sh` = a shell **inside** the running container; `exit` to leave

The six commands you will use every day

Command	Means
`docker run`	Create a container from an image and start it
`docker ps`	What's running? (`-a`: including stopped)
`docker logs`	What has it printed? **Look here first.**
`docker exec`	Run a command inside it
`docker stop`	Stop it
`docker rm`	Delete it

```
docker stop web
docker rm web
```

Act 2

"Build your own box"

Steps 3 to 5: a Dockerfile, an AI that drafts it, and a crash

Step 3: ask an AI for a Dockerfile

In Copilot Chat (**Ask** mode), attach `app.py` and `requirements.txt`, then:

```
Write a Dockerfile for this Flask app. Use Python 3.12.
It listens on port 8000.
Add a comment above every instruction explaining
what it does, so students can understand every command.
```

Read it. Can you explain every line? Check each comment is right; add a `# why:` where it only says what. A line you can't explain gets looked up, or removed.

One I saved earlier: a Dockerfile is a list of steps

```
FROM python:3.12-slim
WORKDIR /app
COPY . .
CMD ["python", "app.py"]
```

Line	Means
`FROM`	Start from the official slim Python 3.12 image
`WORKDIR`	Work in `/app` inside the box
`COPY . .`	Copy my files in
`CMD`	When the container starts, run this

Step 3: build it, run it… and it stops

```
docker build -t novastack-menu:0.1 .
docker run -d --name menu -p 8000:8000 novastack-menu:0.1
docker ps
docker ps -a
docker logs menu
```

```
menu   Exited (1) 2 seconds ago

ModuleNotFoundError: No module named 'flask'
```

Paused here. It worked on my laptop. Why not in the box?

Mentimeter question 2 of 3

The code ran fine on my laptop, but the container **stops within a second**. What is the **most likely cause**?

1. Docker is broken on this laptop
2. Something the code needs was on my laptop but was never put in the box
3. The code needs a faster computer
4. The website address is wrong

menti.com
code **2590 7185**

Question 2: the answer is 2, the box started empty

- Flask is installed on **my laptop**, not in the box
- A container has **only what the image gives it**, plus what you `COPY` in
- "It works on my machine" means my machine has something yours doesn't
- A **Dockerfile** is where you write that something down

> That one line of the log told us exactly what to do. **Look at the log first.**

Step 4: read the log, then ask the AI to read it

1. Predict first: "I think the problem is ___."

2. Give the AI evidence: attach the Dockerfile **and paste the log**. Attaching a file doesn't send your terminal.

```
This container exits straight away. Here are the Dockerfile and
the log. What is wrong, and what is the smallest change that fixes it?
Log: <paste the lines from docker logs menu>
```

3. Compare its answer with your prediction. Does it quote the error from the log?

Step 4: fix the box and prove it

Add one line after `COPY . .`:

```
RUN pip install -r requirements.txt
```

```
docker rm menu
docker build -t novastack-menu:0.2 .
docker run -d --name menu -p 8000:8000 novastack-menu:0.2
curl localhost:8000/menu
```

Open **localhost:8000**: the same page, now **Running on Linux via Docker**.

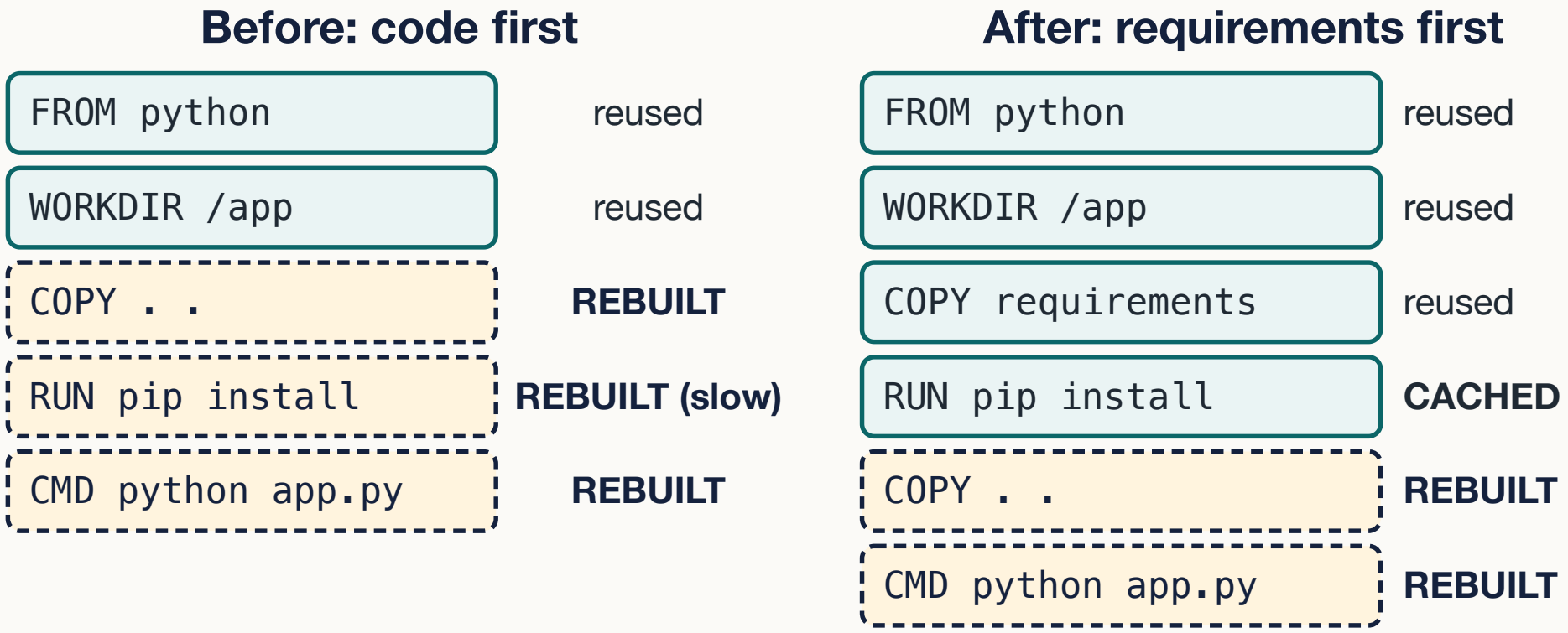
- `RUN` happens when you **build** the image; `CMD` when you **run** a container

Three habits for working with an AI

Role	Prompt	How we proved it
Author	"Write a Dockerfile for this app"	We built it and ran it
Reader	"Here are the file and the log. What is wrong?"	The page loaded
Reviewer	"Review this. At most three problems"	We timed the rebuild (next)

1. Give it **evidence**, not a guess
2. Ask for a **short list**
3. **Prove it** with a command. Docker is the judge.

Step 5: now make it quick



Every instruction is a **layer**. Docker reuses a layer only if it **and everything above it** are unchanged.

Step 5: the AI reviews, we prove one change

```
Review this Dockerfile. Give me at most three problems,
most important first.
```

Typical answers: requirements first · `.dockerignore` · not root · pin the image. We apply the **caching** one: install **before** copying the code.

```
docker build -t novastack-menu:0.4 .
# edit the heading in templates/index.html, then:
docker build -t novastack-menu:0.5 .
```

About 3 s with the code copied first (Flask reinstalled) → **about 0.6 s** now (`CACHED`).

Mentimeter question 3 of 3

Predict: you delete a container that ran a **database**, then start a new one from the **same image**. Where is your data?

1. Still there: it was saved in the image
2. Gone, unless it was kept in a volume
3. Still there, once the new container has restarted
4. In the other containers made from that image

menti.com
code **2590 7185**

Question 3: the answer is 2, gone without a volume

Option	Why
Still there, saved in the image	An image is **read-only**: a running container never writes to it
2. Gone, unless in a volume (correct)	**A container's own files are deleted with the container**
Still there after a restart	A new container starts from the image again
In the other containers	Each container's changes are private (Question 1)

> A **volume** is storage outside any container. Step 6 gives the database one.

Step 5, one more thing: a secret in the box

```
cp .env.example .env          # passwords for Step 6; Git ignores it
docker build -t novastack-menu:0.6 .
docker run --rm novastack-menu:0.6 cat .env
```

```
POSTGRES_USER=menu
POSTGRES_PASSWORD=change-me     # ...and two more lines
```

Git ignored `.env`. **Docker didn't**: `COPY . .` copied it. Add a **`.dockerignore`** (like `.gitignore`, for builds), rebuild as `0.7`, and `cat .env` says No such file.

Act 3

"The whole kitchen"

Step 6: the menu **and** a database, in one command

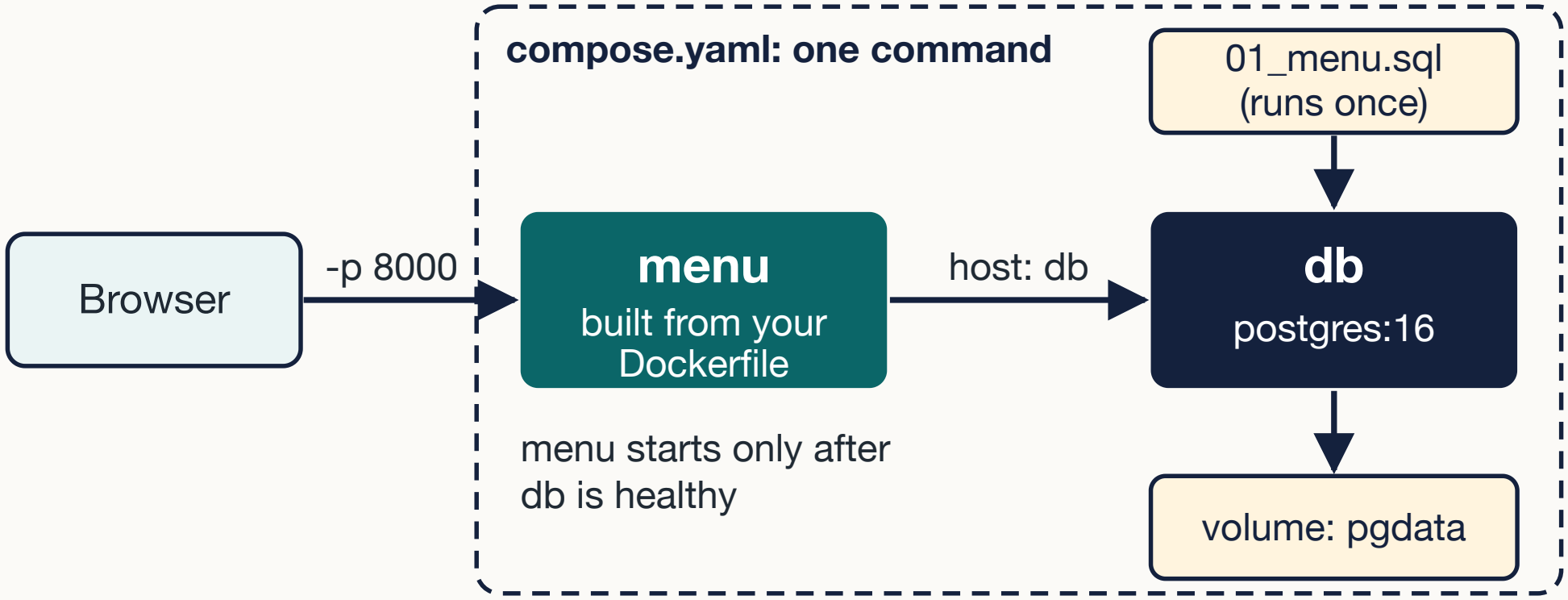
Step 6: why Compose? Two boxes by hand

```
docker network create menu-net
docker volume create pgdata
docker run -d --name db --network menu-net --env-file .env \
  -v pgdata:/var/lib/postgresql/data postgres:16   # + the start-up script
# ...wait until the database is ready...
docker build -t novastack-menu:0.5 .
docker run -d --name menu --network menu-net -p 8000:8000 \
  novastack-menu:0.5                          # + its settings
```

Five commands, a dozen settings, an order and a wait, typed the same by **everyone**. One typo: "it works on my machine" again.

Compose writes it all down in **one file**, `compose.yaml`, committed with the code. Then **one command** starts the lot.

Step 6: one command, two containers



Services (`menu`, `db`) on a private **network**, found **by name**; a **volume** keeps the data; a **health check** says when `db` is ready.

Step 6: start the whole stack

```
docker compose up -d
docker compose ps
docker compose exec db psql -U menu -c "SELECT count(*) FROM items;"
docker compose down
```

```
 count
-------
    18
```

- Postgres 16 ran **without being installed**; the 18 rows came from `01_menu.sql`
- Containers find each other **by service name** (`db`), not `localhost`
- `down` keeps the data; `down -v` deletes it

Habits that save you later

Habit	Why
No secrets in images (`.env` out of Git and out of the image)	A scan of **337,171** images found **8.5%** held valid secrets, such as private keys (Dahlmanns et al., 2023)
Pin versions (`python:3.12-slim`, `flask==3.1.3`), tag releases, avoid `latest`	**34%** of a sample of 560 GitHub projects' Dockerfiles failed to build (Cito et al., 2017)
Don't run as root (`USER`)	Less power if the app is ever broken into
`.dockerignore`	Keeps `.git` and `.env` out, and builds faster

Gotchas to know before the labs

- **Mac: port 5000 is taken** by AirPlay Receiver. Use 8000, 8001 or 8080
- **Apple Silicon builds `arm64` images**; most cloud servers are `amd64`. An unlucky push gives `exec format error` (Demo 3, Step 8)
- **Docker Hub limits downloads** per address: run `docker login`, and pre-pull
- **Windows:** use **PowerShell**, not Git Bash, for `docker exec -it`
- **Disk full?** `docker system df`, then clean up what you don't need

Epilogue: it runs on staging

> **Tom:** "It runs on staging. Python 3.9 and no uv, and I didn't have to touch a thing. 📦 "
>
> **Jordan:** "Good. Now: how do we make sure **nobody breaks it again**? Every change, tested and checked, before it ships. That's the next chapter."

Before the labs

- **Exercise ORD-371** is Part 0 of every lab: finish a Dockerfile, then fix a page that won't load. It meets the `127.0.0.1` bug
- **Beginner:** Ship the Menu · **Intermediate:** Make It Production-Shaped · **Advanced:** App and database with Compose
- Every step is written up in `lectures/05-docker/demo/`
- Run `docker login` and stop containers you don't need (`docker ps -a`)
- Lecture 6 (SQL) starts from the **database you started in Step 6**

Two short forms, please (about 8 minutes)

1. Attendance and survey

University sign-in · about 5 minutes

2. Anonymous feedback

No names recorded · about 2 minutes

Can't scan? Both links are on Moodle. Taking part in the research is **voluntary** and doesn't affect your place on the programme. Every question has an **N/A** option.

Thank you

Questions?

Remember: "it works on my machine" is a sentence that ends with "…because of something I forgot to write down."

Appendix: not today, but you will meet them

Name	One line
Kubernetes	Runs many containers across many machines. Often listed with Docker in job ads
Docker Swarm	Docker's older built-in clustering: almost never asked for
Podman	A Docker-compatible alternative, a free choice if your machine forbids Docker Desktop
Hardened images, signing, SBOMs	Supply-chain security: later in the programme

> At graduate level, employers expect you to **run and build** a container, not to design a cluster.